

FlowShield Project Plan: Hackathon Application Division & Implementation Guide

Project Overview: FlowShield is an embedded financial resilience engine that helps informal workers manage unpredictable income using real-time earnings data. This document outlines the distinct responsibilities, what to build, and how to build it for each of our 5 team members to successfully deliver the MVP.

Member 1: Authentication & User Management

What to do:

- Build the user registration and login system for both workers and partners.
- Implement JWT-based authentication and secure session management.
- Create worker and partner profiles with role-based access control.
- Develop basic user and work history interfaces.

How to do it:

- **Tech Stack:** Python, FastAPI, PostgreSQL (Supabase recommended), JWT, secure password hashing.
- **Database Schema:** Create and manage the users, workers, and partners tables.
- **APIs to Build:** Develop endpoints for POST /auth/register, POST /auth/login, and GET /workers/{id}.
- **Security:** Ensure role-based authorization is enforced on API routes and no sensitive data is exposed.
- **Phase 2 Task:** Complete this during the "Foundation" phase to unblock the rest of the team.



AI Integration & Handoff Guidelines

- **Data Anonymization Pipeline:** You must generate and store deterministic, anonymous UUIDs for all users so the AI engine (Member 3) never ingests Personally Identifiable Information (PII) during scoring or training.
- **Stateful Consent Tokens:** Embed AI data processing consent flags directly into the JWT payloads. This ensures the AI microservices can instantly validate if a user's data can be modeled without querying the database.

Member 2: Income & Transaction System

What to do:

- Create the Earnings API to record daily/weekly/monthly earnings and work logs.
- Track payout and transaction history for workers.
- Perform income calculations and aggregations.
- Develop a simulated partner earning-data API to mock incoming platform data.

How to do it:

- **Tech Stack:** Python, FastAPI, PostgreSQL.
- **Database Schema:** Create and manage the work_logs, earnings, and transactions tables.
- **APIs to Build:** Develop endpoints for POST /earnings and GET /workers/{id}/earnings.
- **Logic:** Write aggregation queries to sum daily/weekly income and calculate baseline earning metrics.
- **Integration:** Ensure Member 3 (AI/Stability Engine) can easily query this structured data for scoring.



AI Integration & Handoff Guidelines

- **Time-Series Normalization:** Standardize all income records using strict ISO-8601 timestamps and a unified baseline currency format to prevent feature-skew in the AI's volatility calculations.
- **Imputation Readiness:** Design API responses to explicitly flag missing data periods (rather than omitting them entirely). This ensures the AI engine can apply consistent data imputation (e.g., zero-fill vs. forward-fill) for sparse income arrays.

Member 3: AI / Stability & Risk Engine

What to do:

- Design and implement the FlowShield Stability Score (0-100).
- Analyze income volatility and detect recent income trends or anomalies (e.g., sudden drops).
- Classify workers into risk tiers: Stable (70-100), At Risk (40-69), or Critical (0-39).
- (Optional) Add weather disruption as an external risk factor.

How to do it:

- **Tech Stack:** Python, Pandas, NumPy, Scikit-learn (if predicting).
- **Database Schema:** Manage the risk_scores table.
- **Algorithms:** Implement the rule-based MVP score: 30% Income Consistency + 25% Work Frequency + 20% Recent Income Trend + 15% Savings Buffer + 10% External Risk Factors.
- **APIs to Build:** Develop endpoints for GET /workers/{id}/stability-score and GET /workers/{id}/risk.
- **Testing:** Unit-test the scoring logic extensively using a shared sample dataset.



AI Integration & Handoff Guidelines

- **Explainability Logging:** Log all exact feature weights, standard deviations, and intermediate metric calculations for every generated score. This ensures downstream debugging and provides the Frontend (Member 5) with explainability contexts.
- **Bounded API Contracts:** Guarantee strict schema adherence—stability scores must always evaluate to a 0-100 float, and risk tiers must map precisely to standardized Enum strings to prevent breaking the state machines in the Protection Engine.

Member 4: Financial Protection Engine

What to do:

- Build the Emergency Pot logic: automatically allocate a small percentage of high-income earnings to a reserve.
- Create automatic savings rules and income-smoothing logic.
- Determine eligibility for emergency support, micro-credit, and insurance based on stability indicators.
- Log the history of financial actions and safety recommendations.

How to do it:

- **Tech Stack:** Python, FastAPI.
- **Database Schema:** Manage the emergency_pots, credit_assessments, and notifications tables.
- **APIs to Build:** Develop endpoints for POST /emergency-pot/contribute, GET /emergency-pot/{worker_id}, and GET /credit/{worker_id}/eligibility.
- **Logic:** Listen to Member 3's risk state changes. If a worker hits "High Risk" due to an income drop, trigger the policy engine to calculate available credit or release emergency funds.



AI Integration & Handoff Guidelines

- **Decoupled Logic Architecture:** Treat the AI's risk scores and states strictly as read-only event triggers. The financial calculation mechanics must not be coupled to the AI engine's internal states.
- **Graceful Degradation:** Implement robust fallback safety-net rules. If the AI engine experiences latency, webhook failure, or downtime, the system must automatically apply default, conservative emergency protection algorithms rather than blocking critical user transactions.

Member 5: Frontend, Dashboards & SDK

What to do:

- Build a clean, mobile-first Worker Dashboard displaying Profile, Stability Score, Income Trend, Emergency Fund, and Recommendations.
- Build the Partner Dashboard displaying workforce distribution (Stable/At Risk/Critical), income volatility, and at-risk worker lists.
- Create interactive charts and handle UI notifications.
- Develop the embedded SDK/widget interface and integrate the frontend with the backend APIs.

How to do it:

- **Tech Stack:** React + Vite, Tailwind CSS + shadcn/ui, Recharts.
- **APIs to Consume:** Connect to GET /dashboard/worker/{id}, GET /dashboard/partner/{id}, and other data endpoints.
- **Design Principles:** Maintain a financial-safety visual language. Prioritize one clear status, one explanation, and one recommended action per screen.
- **Deployment:** Deploy the frontend application using Vercel.



AI Integration & Handoff Guidelines

- **Latency UX Mitigation:** Implement skeleton loaders specifically tailored for AI-dependent widgets (like the Risk Score and Predictions) to successfully manage user expectations during potential inference latency.
- **AI Transparency:** Clearly label insights, trends, and risk classifications as "AI-Generated" or "Model-Driven." You must provide intuitive UI tooltips that briefly explain how the AI arrived at its conclusions (by parsing Member 3's explainability logs).